MACRO RIDES TECHNICAL EVALUATION REPORT

Zone Boundary & Dynamic Route Corridor Visualization Tool

Candidate Name:	Shaswat (shaswat22)	**Submission Date:**	June 24, 2026
Project Domain:	Hyperlocal EV Mobility Solution	**Evaluation Period:**	24 June 2026 – 01 July 2026

1. Executive Summary & Core Objectives

In hyperlocal electric vehicle (EV) mobility networks, minimizing rider response latency and maximizing route-based batch matching are critical operational objectives. This technical dossier presents the architecture and algorithmic framework of an interactive, client-side geospatial visualization platform developed explicitly to satisfy the Macro Rides technical assignment guidelines.

The developed engine supports real-time simulated route updates, constructing an adjustable physical buffer corridor around active vehicles. It maps continuous geographic bounding boundaries into discrete, hierarchical spatial index coordinates, screening passenger hotspot eligibility in constant time ($O(1)$). Designed for optimal portability, the system runs as a serverless Single Page Application (SPA) driven by stable CDN script references without requiring server infrastructure.

2. Architectural & Library Stack Specification

The technical solution isolates components cleanly across a single integrated web viewport. Responsibilities are divided into three core client modules:

Framework Component	API Entry-Points Used	Operational Responsibility & Performance Focus
Leaflet.js (v1.9.4)	`L.map()`, `L.tileLayer()`, `L.polygon()`	Manages map graphics, tracks driver state, handles click capture events for

		route creation, and updates elements dynamically to show target location contexts.
Turf.js (v6)	`turf.lineString()`, `turf.buffer()`, `turf.lineChunk()`	Computes continuous spherical vector geometry, slices polyline networks into small animation units, and draws precise metric catchment buffers.
Uber H3-js (v4.1.0)	`h3.polygonToCells()`, `h3.latLngToCell()`, `h3.cellToBoundary()`	Transforms geographic space into discrete indexes. It processes polyline rings into hexadecimal sets, resolving complex containment checks through fast hash matching.

3. Algorithmic Motivation & Complexity Optimization

Traditional geospatial lookup models rely on checking continuous spatial coordinates. For an environment containing N active drivers and M stationary pickup demands, calculating boundaries

using Haversine equations forces a runtime complexity of $O(N \times M)$. This creates severe computational bottlenecks on mobile devices during large stress tests.

This engine avoids continuous distance loops by using discrete spatial quantization. As the driver marker progresses along its path steps, the mathematical flow executes as follows:

1. **Spatial Discretization:** The continuous buffer ring shape generated by Turf.js is transformed into a set of active H3 grid tokens using the `polygonToCells` API:

   ```
   C_buffer = h3.polygonToCells(polygonCoords, h3Resolution);
   ```

2. **Hash Table Conversion:** The array is wrapped inside a native JavaScript lookup container to enable rapid checks:

   ```
   const bufferSet = new Set(bufferedH3Cells);
   ```

3. **Constant-Time Lookups:** When checking pickup locations, their points are mapped to an H3 index using `latLngToCell` and matched against the set, lowering runtime complexity per point to $O(1)$:

   ```
   const isEligible = bufferSet.has(h3Index);
   ```

4. Technical Assignment Verification Metrics

The platform delivers comprehensive coverage across all strict assignment evaluation requirements:

- **350 Meter Route Corridor:** Satisfied. Controlled by an interactive slider, allowing live tests scaling from 100m to 1000m (defaulting at the specified 350m line).
- **Zone Boundaries & H3 Indexing:** Satisfied. Renders explicit hexagonal cell perimeters dynamically via `h3.cellToBoundary`, visually tracking active matching zones.
- **Simulated Route Updates:** Satisfied. Slices coordinates into 200 progressive steps via `turf.lineChunk`, driving smooth updates along the timeline every 250ms.

5. Data Processing Pipeline Architecture

The runtime pipeline maps events directly from user actions down to visual map modifications:

```
[User Map Click / Text Input] ──> [OSM Nominatim Gateway Centering] │ ▼ [Polyline Construction & turf.lineChunk Slicing (200 Frames)] │ ▼ [Interval Timer Step Increment Update (250ms)] │ ▼ [Turf.js Spherical Meter Buffer Generation] │ ▼ [Uber H3 Indexing via h3.polygonToCells()] │ ▼ [O(1) Set Lookups: Yellow = Active Eligible | Grey = Inactive] │ ▼ [Leaflet Vector Map Canvas Layer Redraw]
```

6. Interactive Application Controls

1. **Click-to-Route Workspace:** Users can tap multiple points directly on the map surface to dynamically construct paths.
2. **Jump to Location:** Communicates with OpenStreetMap Nominatim endpoints to teleport the simulation engine instantly to new centers (e.g., IIT Kanpur).
3. **Variable Resolution Selector:** Toggles between Resolution 8 (~700m bounds), Resolution 9 (~100m localized), and Resolution 10 (~40m precision) to benchmark spatial grid performance.

> **Official Submission Repositories:**
> • **Production Live Demo:** https://macro-rides-mobility.netlify.app
> • **Source Code Repository:** https://github.com/shaswatgithub/macro-rides-mobility

7. Implementation Code Excerpt (Core Mechanics)

```
// Core logic capturing the buffer conversion and O(1) matching cycle
const bufferGeoJSON = turf.buffer(turf.point(currentLngLat), bufferRadius, { units:
"meters" });
const polygonCoords = bufferGeoJSON.geometry.coordinates[0].map(coord => [coord[1],
coord[0]]);

let bufferedH3Cells = h3.polygonToCells(polygonCoords, h3Resolution);
const bufferSet = new Set(bufferedH3Cells);

mockPickups.forEach((p) => {
    const h3Index = h3.latLngToCell(p.coords[0], p.coords[1], h3Resolution);
    const isEligible = bufferSet.has(h3Index);

    L.circleMarker(p.coords, {
        radius: isEligible ? 6.5 : 4.5,
        color: isEligible ? "#ca8a04" : "#9ca3af",
        fillColor: isEligible ? "#facc15" : "#e4e4e7"
    }).addTo(pickupLayerGroup);
});
```